

Atelier : Migration d'une base de données via Kafka connect.

Préparation PostgreSQL

Dans pgAdmin on vérifie :

SHOW wal_level;

Le résultat attendu doit être **logical**. Si ce n'est pas le cas, modifiez le fichier **postgresql.conf** (**SHOW config_file;**) .

wal_level = logical

max_replication_slots = 10

max_wal_senders = 10

Redémarrer **PostgreSQL**

*Cette commande **SHOW wal_level** permet d'afficher le niveau de détails enregistré dans le WAL.*

*Dans **postgresql**, le **WAL** est le journal transactionnel, chaque action-événement (insert-update-delete, etc) est d'abord enregistré dans ce fichier avant d'être écrit sur la table. Le **WAL** sert :*

- *la reprise après crash*
- *la réplication*
- *l'audit des modifications*

*Le **WAL** a donc plusieurs niveau :*

***minimal** : Le **WAL** écrit juste le minimum pour assurer la récupération.*

***replica** : Le **WAL** donne suffisamment d'information pour la réplication mais postgres ne sait pas ce qui a changé au niveau des détails métiers (insert - update etc).*

***logical** : C'est le plus haut niveau de détail. Ici le **WAL** contient suffisamment d'informations pour reconstruire la base de données table par table. C'est ici exactement ce que lit **Debezium**.*

Création des bases dans pgAdmin

- **source_db**
- **target_db**

```
CREATE TABLE customers (  
    id SERIAL PRIMARY KEY,  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
    email VARCHAR(200),  
    updated_at TIMESTAMP DEFAULT now()  
);
```

```
INSERT INTO customers(first_name, last_name, email)  
VALUES  
(  
'Didier', 'MALEMBE', 'didier@malembe.com'),  
(  
'Pablo', 'Depitcho', 'pablo@test.com');
```

```
ALTER TABLE customers REPLICA IDENTITY FULL;
```

on vérifie : `SELECT * FROM customers;`

Préparation des plugins Kafka Connect

Construire l'arborescence :

```
C:\kafka\plugins\  
C:\kafka\plugins\debezium-postgres\  
C:\kafka\plugins\debezium-jdbc\
```

Récupérer les dépendances

Debezium PostgreSQL Connector
Debezium JDBC Sink Connector

<https://debezium.io/releases/3.4/>

PostgreSQL JDBC Driver

https://jdbc.postgresql.org/download/?utm_source=chatgpt.com

Configurer Kafka Connect Standalone

Créer le fichier **connect-standalone.properties**

```
bootstrap.servers=localhost:9092
key.converter=org.apache.kafka.connect.json.JsonConverter
value.converter=org.apache.kafka.connect.json.JsonConverter

key.converter.schemas.enable=true
value.converter.schemas.enable=true

offset.storage.file.filename=C:/kafka/connect.offsets
offset.flush.interval.ms=10000

plugin.path=C:/kafka/plugins
```

Création du fichier source connector

Créer le fichier **postgres-source.properties**

```
name=postgres-source
connector.class=io.debezium.connector.postgresql.PostgresConnector
tasks.max=1

database.hostname=localhost
database.port=5432
database.user=postgres
database.password=mdp
database.dbname=source_db

topic.prefix=demo
plugin.name=pgoutput
slot.name=demo_slot
publication.autocreate.mode=filtered
schema.include.list=public
table.include.list=public.customers

snapshot.mode=initial
```

Créer le fichier sink connector

postgres-sink.properties

```
name=postgres-sink
connector.class=io.debezium.connector.jdbc.JdbcSinkConnector
tasks.max=1

connection.url=jdbc:postgresql://localhost:5432/target_db
connection.username=postgres
connection.password=mdp

topics=demo.public.customers

insert.mode=upsert
delete.enabled=true

primary.key.mode=record_key
primary.key.fields=id

schema.evolution=basic
table.name.format=customers
```

Démarrer Kafka

```
cd C:\kafka
```

```
bin\windows\kafka-server-start.bat config\kraft\server.properties
```

Démarrer Kafka Connect Standalone

```
cd C:\kafka
```

```
bin\windows\connect-standalone.bat connect\connect-standalone.properties
connect\postgres-source.properties connect\postgres-sink.properties
```

Kafka Connect démarre sur les deux connecteurs.

Vérification

curl -X POST <http://localhost:8083/connectors>

curl http://localhost:8083/connectors/postgres-source/status

curl http://localhost:8083/connectors/postgres-sink/status